C0 Start here: foundations for Fabric

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track C: Microsoft Fabric 2

C0 Start here: foundations for Fabric 3

C0-L1 Data platform vocabulary, files, tables, lakes and warehouses 3

C0-L2 Power BI basics, reports, semantic models and measures 6

C0-L3 Two engines, one table, Spark notebooks and the SQL analytics endpoint 10

Track C best-practice checklist 14

Appendix: solutions and answer keys 15

Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

C0 Start here: foundations for Fabric

Roadmap: Prerequisite for all months. Data platform vocabulary, Power BI semantic models, and the Spark and SQL engines side by side.

C0-L1 Data platform vocabulary, files, tables, lakes and warehouses

Concept

Fabric lessons assume a shared vocabulary. Here it is, platform first, then how Fabric maps onto it.

Files and formats.

- **CSV** is plain text. It stores no types, so a reader guesses them, and a postcode `01234` can come back as the number `1234`.
- **Parquet** stores data by column, with each column's type, and compresses it. Reading two columns of a wide table reads only those two.
- **Delta** is a folder of Parquet files plus a **transaction log**, `_delta_log`. The log turns files into a table: each write is a numbered version, readers never see a half-finished write, and you can read an older version.

Places for data.

- A **data lake** holds files of any shape, cheaply. Structure is applied when you read.
- A **data warehouse** holds tables with a fixed schema, queried with SQL.
- A **lakehouse** keeps tables in an open format, such as Delta, in a lake, so a Spark notebook and a SQL engine read the same data.

Storage and compute are separate. Data sits once in storage. Engines, such as Spark, SQL and Power BI, bring compute to it when they run.

How data moves. Batch processing handles a delivery at a time, such as a nightly file. **Streaming** handles events as they arrive, seconds apart. With **ELT**, you load raw data first, then transform it inside the platform. Many teams name the stages **bronze** (raw, as received), **silver** (cleaned) and **gold** (aggregated for reports).

In Fabric, all of this lives in **OneLake**, one logical lake for the whole organisation (GA). Lakehouse tables are Delta tables (GA). Spark reads them,

and the **SQL analytics endpoint** (GA) queries the same tables with T-SQL. Lessons C1-L2 and C2-L1 go deeper.

Why it matters

A team exported its gold tables to CSV for a reporting tool. Store codes such as `00042` became `42`, and the join to the store list silently matched nothing for 300 stores. Reading the Delta table directly, with its types intact, avoided both the copy and the broken join.

Worked example: weak code

```
  1 # Export gold to CSV, then read it back for a report
(1) gold.to_csv("gold_stores.csv", index=False)
(2) report = pd.read_csv("gold_stores.csv")
```

1. **Line 2.** A second copy of the data, in a format that stores no types.
2. **Line 3.** The reader guesses types: store code 00042 comes back as 42.

Worked example: strong code

```
  1 # Read the one Delta table that every engine shares
(1) report = spark.read.table("gold.store_sales")
```

1. **Line 2.** No copy, and the column types travel with the table.

Keep data once, in a typed table format such as Delta, and let each engine read it, instead of exporting copies to untyped files.

Recap card

- **CSV, Parquet and Delta** CSV: text, no types. Parquet: columnar, typed, compressed. Delta: Parquet files plus a transaction log with versions.
- **Lake, warehouse, lakehouse** Lake: files, structure on read. Warehouse: typed tables for SQL. Lakehouse: open tables in a lake, read by Spark and SQL.
- **ELT and medallion stages** Load raw first, transform inside the platform: bronze (as received), silver (clean), gold (aggregated).
- **Fabric's names for these** OneLake is the one lake (GA). Lakehouse tables are Delta (GA). The SQL analytics endpoint queries them with T-SQL (GA).

Practice

Exercise 1. Multiple choice (level 1)

What makes a folder of Parquet files a Delta table?

- a) A transaction log that records every write as a numbered version
- b) A different file extension
- c) Storing the data as CSV
- d) Running a SQL engine on top

Why?

- r1) The log tells readers which files form each version, so writes are all-or-nothing and old versions stay readable.
- r2) The log compresses the data.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0 with pyarrow)

What you need to know.

The code uses pandas, Python's table library, and the `io` module:

- `pd.DataFrame({"postcode": ["01234"]})` builds a one-column table.
- `df.to_csv(index=False)` writes it as CSV text. `pd.read_csv(...)` reads CSV back.
- `df.to_parquet(buffer)` writes it as Parquet. `pd.read_parquet(buffer)` reads Parquet back.
- `io.StringIO(text)` and `io.BytesIO()` are in-memory stand-ins for a text file and a binary file. `buffer.seek(0)` rewinds to the start before reading.
- `["postcode"].iloc[0]` takes the first value of the column.

```
import io

import pandas as pd

df = pd.DataFrame({"postcode": ["01234"]})
csv_text = df.to_csv(index=False)
buffer = io.BytesIO()
df.to_parquet(buffer)
buffer.seek(0)
print(pd.read_csv(io.StringIO(csv_text))["postcode"].iloc[0],
    pd.read_parquet(buffer)["postcode"].iloc[0])
```

Exercise 3. Fill the gap (level 2)

Complete the definitions.

```
A Delta table is a folder of _____(1) files plus a transaction log.
A _____(2) keeps open-format tables in a lake, so Spark and SQL read the same data.
With _____(3), you load raw data first and transform it inside the platform.
```

Exercise 4. Put in order (level 2)

Put the stages of a typical ELT flow for a nightly delivery in order.

- Clean, rename and type it into a silver table.
- A semantic model and its reports read the gold tables.
- Load it into a bronze table, as received, with audit columns.
- The delivery file lands in the lake, unchanged.
- Aggregate it into gold tables for reporting.

Exercise 5. Architecture design (level 3)

Pick the best fit for each need.

Options: Files in a data lake, Delta tables in a lakehouse, Streaming ingestion, A nightly CSV export.

1. Keep supplier PDFs and raw JSON exports cheaply, structure unknown
2. A table that a Spark notebook and a SQL report must both read, with types intact
3. Vehicle readings that must reach a dashboard within seconds

Sources

- A CSV round trip turns the text 01234 into 1234 in pandas 3.0, while a Parquet round trip keeps 01234. (verified: Run on pandas 3.0.2 with pyarrow 22.0.0 and pandas 3.0.6 with pyarrow 26.0.0.)
- OneLake, Lakehouse (Delta tables) and the SQL analytics endpoint are GA. (search-only: VERSIONS.md Fabric feature status, checked on Microsoft Learn in September 2026.)

C0-L2 Power BI basics, reports, semantic models and measures

Concept

Power BI separates three layers. Keep them apart in your head.

- **Data** lives in tables at the source, such as Delta tables in a lakehouse.

- A **semantic model** describes that data for analysis: which tables it uses, how they relate, and which calculations exist.
- A **report** holds pages of **visuals**, such as charts and tables, that query the model.

Tables and relationships. A model usually has a **fact table**, one row per event or response, holding numbers and keys, and **dimension tables** that describe those keys, such as region or brand. A relationship links each dimension to the fact table, one-to-many, so a filter on a region narrows the facts to that region. Lesson C6-L1 builds this properly as a star schema.

Columns and measures. A **calculated column** is worked out once per row and stored. A **measure** is worked out when a visual asks, for whatever the visual currently shows. That difference matters for ratios:

```
1 Weighted reach % = DIVIDE ( SUM ( fact[reach] ), SUM ( fact[base] ) )
```

This measure divides two totals, so it is right at every level: a region, a wave or everything. An average of per-row percentages weights a row with 2 respondents the same as a row with 2,000.

Filter context. Each visual cell evaluates a measure under the filters that apply to it: slicers, the row and column headers, and page filters. The same measure gives North's figure in North's row and the total in the total row. Lesson C6-L2 shows how to change that context on purpose.

`DIVIDE` returns blank when the denominator is 0 or blank.

Storage modes. Import copies the data into the model and refreshes it on a schedule. **DirectQuery** sends each query to the source. **Direct Lake** (GA in Fabric) reads Delta tables in OneLake directly, without an import copy (lesson C6-L3).

Why it matters

A dashboard showed "average reach %" as the average of a calculated column, reach divided by base for each region-wave row. Small regions counted as much as large ones, and the national figure read 70% where the true weighted reach was 83%. A measure dividing the summed reach by the summed base gives 83% at every level, including the total row.

Worked example: weak code

```
1 Calculated column: fact[reach_pct] = fact[reach] / fact[base]
2 Visual value: Average of fact[reach_pct]
```

1. **Line 1.** Worked out once per row and stored, before any filter applies.

2. **Line 2.** Averages the row percentages, so a tiny region counts as much as a large one.

Worked example: strong code

```
  1 Weighted reach % =
❶ DIVIDE ( SUM ( fact[reach] ), SUM ( fact[base] ) )
```

1. **Line 2.** A measure: two totals divided, evaluated for whatever each visual cell shows. DIVIDE returns blank for a zero base.

Put ratios in measures that divide totals, and let filter context decide what each total covers.

Recap card

- **Data, semantic model, report** Data sits in source tables. The model describes tables, relationships and measures. Reports hold visuals that query the model.
- **Calculated column against measure** A column is worked out per row and stored. A measure is worked out when a visual asks, in that visual's filter context.
- **Ratios in Power BI** A measure dividing totals: DIVIDE (SUM (reach), SUM (base)). Never average per-row percentages.
- **Import, DirectQuery, Direct Lake** Import copies and refreshes. DirectQuery queries the source each time. Direct Lake (GA) reads OneLake Delta tables directly.

Practice

Exercise 1. Multiple choice (level 1)

Which layer do you change to add a new calculation that every report can use?

- a) The semantic model, by adding a measure
- b) One report's visual
- c) The source Delta table, by adding a column for each possible filter
- d) The report's page filter

Why?

- r1) Reports query the model, so a measure defined there is available to all of them.
- r2) The model is where the data is stored.

Exercise 2. Predict the output (level 2)

The fact table has two rows: reach 1 with base 2, and reach 9 with base 10. What do the two total cells show, rounded to two decimals? Answer as two numbers: the average of the calculated column, then the measure.

```
Calculated column: fact[reach_pct] = fact[reach] / fact[base]
Total cell 1: Average of fact[reach_pct]
Total cell 2: DIVIDE ( SUM ( fact[reach] ), SUM ( fact[base] ) )
```

Exercise 3. Fill the gap (level 2)

Write the weighted reach measure so a zero base gives a blank.

```
Weighted reach % =
_____(1) ( _____(2) ( fact[reach] ), SUM ( fact[base] ) )
```

Exercise 4. Architecture design (level 2)

A survey model has these tables. Classify each one.

Options: Fact table, Dimension table.

1. fact_response: one row per respondent per wave, with weight, reach and keys
2. dim_region: one row per region code, with the region's name
3. dim_wave: one row per wave, with its fieldwork dates

Exercise 5. Put in order (level 3)

A report shows weighted reach for North. Put the path from data to that number in order.

- The measure divides the summed reach by the summed base for those rows.
- Gold Delta tables hold the responses in OneLake.
- The semantic model uses those tables and relates dim_region to the fact table.
- A slicer on the report page selects North.
- The filter flows from dim_region to the fact rows for North.

Sources

- DIVIDE returns BLANK for a zero or BLANK denominator unless an alternate result is passed. (verified: https://learn.microsoft.com/en-us/dax/best-practices/dax-divide-function-operator (as cited in C6-L2))
- Direct Lake is GA in Fabric; Import and DirectQuery are Power BI storage modes. (search-only: VERSIONS.md Fabric feature status (fundamentals/

direct-lake-overview, 2026-09-02) and Power BI storage mode documentation.)

C0-L3 Two engines, one table, Spark notebooks and the SQL analytics endpoint

Concept

A Fabric lakehouse (GA) stores its tables once, as Delta in OneLake. Two engines read them, and you will use both.

Spark, in a notebook. Fabric notebooks run PySpark and give you a `spark` session. Runtime 2.0 (GA) runs Spark 4.1, and the examples here behave the same on the Spark 4.0 used in Track A. You work with DataFrames:

```
from pyspark.sql import functions as F

df = spark.read.table("respondents")
totals = df.filter(F.col("final_weight") >
    0).groupBy("region").agg(F.sum("final_weight").alias("total"))
totals.show()
```

`filter`, `groupBy`, `agg`, `select` and `orderBy` are **transformations**: they describe a new DataFrame and run nothing. **Actions** run the plan: `show()` prints rows, `count()` counts them and `collect()` returns them to Python as `Row` objects. `groupBy("region").count()` is a shortcut that adds a `count` column. In tests, `spark.createDataFrame(rows, "region STRING, w DOUBLE")` builds a small DataFrame. `spark.sql("SELECT ...")` runs Spark SQL in the same notebook.

Spark also **writes**: `df.write.mode("append").saveAsTable("respondents")` adds rows to a managed table, and `mode("overwrite")` replaces them.

T-SQL, on the SQL analytics endpoint. Every lakehouse gets a SQL analytics endpoint (GA). It answers T-SQL queries over the same tables, so analysts and reports can query with familiar SQL:

```
SELECT region, SUM(final_weight) AS total
FROM dbo.respondents
WHERE final_weight > 0
GROUP BY region;
```

The endpoint is **read-only for table data**. You can create views on it, but `INSERT`, `UPDATE` and `DELETE` against lakehouse tables fail there. Write with Spark, or use a Warehouse when you need T-SQL writes (lesson C2-L3).

Pick by job: Spark for loading and transforming, the SQL endpoint for querying and reporting.

Why it matters

A team planned its nightly load as T-SQL `INSERT` statements on the SQL analytics endpoint, because the analysts knew SQL. The first run failed on every statement, since the endpoint does not write to lakehouse tables. Moving the load to a Spark notebook, and keeping T-SQL for the reports, matched each engine to what it does.

Worked example: weak code

```
1 -- On the SQL analytics endpoint
2 INSERT INTO dbo.respondents (respondent_id, final_weight)
3 SELECT respondent_id, final_weight FROM dbo.staging_w14;
```

1. **Line 2.** The endpoint is read-only for lakehouse table data, so this fails.

Worked example: strong code

```
1 -- Load with Spark, in a notebook:
2 -- spark.read.table("staging_w14").write.mode("append").saveAsTable("respondents")
3 -- Query with T-SQL, on the SQL analytics endpoint:
4 SELECT region, SUM(final_weight) AS total
5 FROM dbo.respondents
6 GROUP BY region;
```

1. **Line 2.** Spark writes the Delta table.
2. **Line 4.** The endpoint reads the same table with T-SQL, without a copy.

One Delta table, two engines: write and transform with Spark, query with T-SQL on the SQL analytics endpoint.

Recap card

- **Two engines over one lakehouse table** Spark in notebooks reads and writes. The SQL analytics endpoint queries with T-SQL and is read-only for table data.
- **Append rows to a lakehouse table from Spark** df.write.mode("append").saveAsTable("respondents"). mode("overwrite") replaces the rows.
- **Spark transformations against actions** filter, groupBy, agg, select describe a new DataFrame. show, count, collect run the plan.
- **Where to put T-SQL writes** In a Warehouse. The lakehouse's SQL analytics endpoint does not write table data.

Practice

Exercise 1. Multiple choice (level 1)

You need to append tonight's delivery to the lakehouse table `respondents`. Where do you write the code?

- a) In a Spark notebook, with `df.write.mode("append").saveAsTable("respondents")`
- b) On the SQL analytics endpoint, with `INSERT INTO dbo.respondents ...`
- c) In a Power BI report
- d) Anywhere: every engine can write lakehouse tables

Why?

- r1) Spark writes lakehouse tables. The SQL analytics endpoint is read-only for table data.
- r2) Spark is faster than SQL.

Exercise 2. Predict the output (level 2)

What does this print? (Spark 4.0 and later)

```
from pyspark.sql import functions as F

df = spark.createDataFrame([("N", 2.0), ("S", 0.0), ("S", 3.0)], "region
   STRING, w DOUBLE")
print(df.filter(F.col("w") >
   0).groupBy("region").count().orderBy("region").collect())
```

Exercise 3. Fill the gap (level 2)

Read the staging table and add its rows to `respondents`, keeping the rows already there.

```
spark.read._____(1)
   ("staging_w14").write.mode("_____(2)").saveAsTable("respondents")
```

Exercise 4. Write the function (level 3)

On the SQL analytics endpoint, write a query that returns each `region` with `total_weight`, the sum of `final_weight`, from `dbo.respondents`. Count only rows with a positive weight, and sort by region.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
SELECT ...
FROM ...
WHERE ...
GROUP BY ...
```

```
5 ORDER BY ...
```

Starter code:

```
1 SELECT *
2 FROM dbo.respondents;
```

Exercise 5. Spot the bug (level 3)

This script runs on a lakehouse's SQL analytics endpoint and fails. Tap the line that cannot work there.

```
1 SELECT COUNT(*) AS rows_before FROM dbo.respondents;
2 INSERT INTO dbo.respondents (respondent_id, final_weight) VALUES ('R9',
    1.0);
3 SELECT COUNT(*) AS rows_after FROM dbo.respondents;
```

Write down the line number of the bug.

Sources

- The E2 code prints [Row(region='N', count=1), Row(region='S', count=1)]; spark.sql runs SQL in the same session. (verified: Run on PySpark 4.0.4 locally. Not run on Fabric Runtime 2.0 (Spark 4.1).)
- The SQL analytics endpoint (GA) is read-only for lakehouse table data and supports views; Lakehouse is GA; Fabric Runtime 2.0 (GA) runs Spark 4.1. (search-only: VERSIONS.md Fabric feature status (data-engineering/lakehouse-sql-analytics-endpoint, runtime), as taught in C2-L3 and C5-L2.)

Track C best-practice checklist

☐ Check the status (GA or preview) of every Fabric feature before you depend on it.

☐ One copy of each dataset. Use shortcuts or mirroring before you copy.

☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.

☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.

☐ Semantic models use star schemas. DAX measures state their filter context.

☐ Direct Lake models are checked for fallback to DirectQuery.

☐ Every Copilot output is read, run on known data and compared against a second method.

☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.

☐ Items live in Git. Changes move through deployment pipelines.

☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

C0-L1 Data platform vocabulary, files, tables, lakes and warehouses

C0-L1 exercise 1

Hints. (1) The data files are Parquet either way. (2) What sits in `_delta_log`? (3) A record of every write.

Answer. a) A transaction log that records every write as a numbered version Because: r1) The log tells readers which files form each version, so writes are all-or-nothing and old versions stay readable.

Delta is Parquet plus a transaction log. The log gives versions, all-or-nothing writes and reads of older versions.

Common wrong answers.

- `b`: Delta data files are ordinary Parquet files. The log beside them makes the table.
- `c`: Delta stores data as Parquet, which keeps types. CSV keeps none.
- `d`: Engines read Delta tables, but the table exists without any engine running.
- `a|r2`: Right part, wrong reason. Parquet does the compressing. The log records versions.

C0-L1 exercise 2

Hints. (1) Which format stores the column type? (2) What does a reader guess for `01234` in a text file? (3) CSV guesses a number. Parquet remembers text.

Answer.

```
1 1234 01234
```

CSV keeps only text, so the reader infers a number and the zero is gone. Parquet records that the column is text and returns it unchanged.

Common wrong answers.

- `01234 01234`: CSV stores no types, so read_csv guesses a number and drops the leading zero.

- `1234 1234`: Parquet stores the column's type, text, so the leading zero survives.

C0-L1 exercise 3

Hints. (1) A columnar, typed file format. (2) Lake plus warehouse. (3) Extract, load, then transform.

Answer.

```
A Delta table is a folder of Parquet files plus a transaction log.
A lakehouse keeps open-format tables in a lake, so Spark and SQL read the same data.
With ELT, you load raw data first and transform it inside the platform.
```

Delta is Parquet plus a log, a lakehouse puts open tables in a lake, and ELT loads before it transforms.

Common wrong answers.

- g1 = `CSV`; g2 = `lakehouse`; g3 = `ELT`: Delta data files are Parquet, which keeps types and compresses by column.
- g1 = `Parquet`; g2 = `warehouse`; g3 = `ELT`: A warehouse holds tables for SQL. Open tables in a lake, shared by Spark and SQL, make a lakehouse.
- g1 = `Parquet`; g2 = `lakehouse`; g3 = `ETL`: ETL transforms before loading. Loading first, then transforming, is ELT.

C0-L1 exercise 4

Hints. (1) Raw first, clean next, summary last. (2) Bronze, silver, gold. (3) Reports read the summary.

Answer.

1. The delivery file lands in the lake, unchanged.
2. Load it into a bronze table, as received, with audit columns.
3. Clean, rename and type it into a silver table.
4. Aggregate it into gold tables for reporting.
5. A semantic model and its reports read the gold tables.

Land, load raw to bronze, clean to silver, aggregate to gold, then report. Each stage keeps the one before it, so you can rebuild after a fix.

Common wrong answers.

- Bronze keeps the delivery as received. Cleaning comes after, in silver, so you can always rebuild from the original.
- Aggregate clean data. Gold builds on silver.

C0-L1 exercise 5

Hints. (1) Structure known or unknown? (2) One copy, several engines. (3) Seconds, not nights.

Answer.

- Unknown structure, kept cheaply: files in a lake.
- One typed table for several engines: Delta in a lakehouse.
- Seconds matter: streaming, not nightly batches.
- A CSV export copies the data and loses its types, so it fits none of these.

Raw, unknown files go to the lake, shared typed tables go to the lakehouse, and seconds-fresh data needs streaming.

Common wrong answers.

- shared = `csv_export`: A CSV export makes a second copy without types. Both engines can read the Delta table directly.
- telemetry = `lakehouse`: A lakehouse table can hold the readings, but getting them there within seconds is a streaming job.

C0-L2 Power BI basics, reports, semantic models and measures

C0-L2 exercise 1

Hints. (1) Which layer do all reports read from? (2) Calculations that adapt to filters are measures. (3) Measures live in the model.

Answer. a) The semantic model, by adding a measure Because: r1) Reports query the model, so a measure defined there is available to all of them.

Reports query the semantic model, so a measure there serves every report and adapts to each visual's filters.

Common wrong answers.

- `b`: A visual-level calculation lives in that one report. Other reports cannot reuse it.
- `c`: A measure adapts to any filter at query time. A column per filter does not scale.
- `d`: A page filter narrows what one page shows. It adds no calculation.
- `a|r2`: Right layer, wrong reason. With Direct Lake or DirectQuery the model stores no data copy. It is shared because reports query it.

C0-L2 exercise 2

Hints. (1) Row ratios: 1 / 2 and 9 / 10. (2) Average them: (0.5 + 0.9) / 2. (3) The measure: (1 + 9) / (2 + 10).

Answer.

```
0.7 0.83
```

The average of row ratios is 0.7, because the small row counts as much as the large one. The measure divides the totals: 10 / 12 = 0.83.

Common wrong answers.

- `0.83 0.83`: The column holds 0.5 and 0.9 per row. Their plain average is 0.7.
- `0.7 0.7`: The measure divides the totals: 10 / 12 is 0.83.

C0-L2 exercise 3

Hints. (1) The DAX function that divides safely. (2) Total the reach the same way you total the base. (3) `DIVIDE` and `SUM`.

Answer.

```
Weighted reach % =
DIVIDE ( SUM ( fact[reach] ), SUM ( fact[base] ) )
```

`DIVIDE` returns blank for a zero denominator, and summing both parts makes the ratio correct at any level of the report.

Common wrong answers.

- g1 = `DIVIDE`; g2 = `AVERAGE`: Divide total by total. An average of reach over a sum of base mixes two different levels.

C0-L2 exercise 4

Hints. (1) Which table holds numbers you add up? (2) Which tables have one row per key? (3) Describing tables are dimensions.

Answer.

- fact_response holds the events and numbers, one row per respondent per wave.
- dim_region and dim_wave describe keys, one row per key, and filter the facts through one-to-many relationships.

The fact table records events with numbers. Dimensions describe the keys and filter the facts.

Common wrong answers.

- region = `fact`: dim_region describes a key, one row per region. It filters the facts, so it is a dimension.
- responses = `dimension`: fact_response holds the numbers you add up, one row per event: a fact table.

C0-L2 exercise 5

Hints. (1) Data, then model, then report. (2) The slicer sets the filter context. (3) Filter first, then calculate.

Answer.

1. Gold Delta tables hold the responses in OneLake.
2. The semantic model uses those tables and relates dim_region to the fact table.
3. A slicer on the report page selects North.
4. The filter flows from dim_region to the fact rows for North.
5. The measure divides the summed reach by the summed base for those rows.

Data sits in tables, the model relates them, the slicer sets the filter, the filter narrows the facts, and the measure computes over the remaining rows.

Common wrong answers.

- The filter narrows the rows first. The measure then evaluates over what is left.
- The model describes tables that already exist at the source.

C0-L3 Two engines, one table, Spark notebooks and the SQL analytics endpoint

C0-L3 exercise 1

Hints. (1) Which engine writes lakehouse tables? (2) What does the SQL analytics endpoint do with INSERT? (3) Spark writes, the endpoint reads.

Answer. a) In a Spark notebook, with `df.write.mode("append").saveAsTable("respondents")` Because: r1) Spark writes lakehouse tables. The SQL analytics endpoint is read-only for table data.

Spark writes lakehouse tables. The SQL analytics endpoint queries them and is read-only for their data.

Common wrong answers.

- `b`: The SQL analytics endpoint does not write lakehouse table data. INSERT fails there.
- `c`: Reports read data. They do not load it.
- `d`: The SQL analytics endpoint is read-only for lakehouse table data.
- `a|r2`: Right place, wrong reason. It is about which engine can write, not speed.

C0-L3 exercise 2

Hints. (1) Which rows survive w > 0? (2) Then count rows per region. (3) collect returns Row objects.

Answer.

```
[Row(region='N', count=1), Row(region='S', count=1)]
```

The filter drops the zero weight, leaving one row per region. `count()` after `groupBy` adds a `count` column, and `collect()` returns `Row` objects.

Common wrong answers.

- `[Row(region='N', count=1), Row(region='S', count=2)]`: The filter runs first and drops the S row with w 0.0.
- `[('N', 1), ('S', 1)]`: collect() returns Row objects, which print with their field names.

C0-L3 exercise 3

Hints. (1) The reader method that takes a table name. (2) Add rows, do not replace them. (3) `table` and `append`.

Answer.

```
spark.read.table("staging_w14").write.mode("append").saveAsTable("respondents")
```

`spark.read.table(...)` reads a lakehouse table, and `mode("append")` adds rows instead of replacing them.

Common wrong answers.

- g1 = `table`; g2 = `overwrite`: overwrite replaces the rows already in the table. The task keeps them: append.

C0-L3 exercise 4

Hints. (1) Start with `SELECT region, SUM(final_weight) AS total_weight FROM dbo.respondents`. (2) Filter rows before grouping: `WHERE final_weight > 0`. (3) Then `GROUP BY region ORDER BY region`.

Answer.

```
1 SELECT region, SUM(final_weight) AS total_weight
2 FROM dbo.respondents
3 WHERE final_weight > 0
4 GROUP BY region
5 ORDER BY region;
```

`WHERE` filters rows, `GROUP BY` makes one row per region, and `ORDER BY` sorts the result. The query only reads, which is all the endpoint allows on table data.

C0-L3 exercise 5

Hints. (1) Which statements only read? (2) Can the endpoint change lakehouse table data? (3) It is read-only for table data.

Answer. The bug is on line 2.

The SQL analytics endpoint is read-only for lakehouse table data, so the INSERT fails. Add the row from a Spark notebook, or keep data that needs T-SQL writes in a Warehouse.

Common wrong answers.

- Line 1: Reading a lakehouse table on the endpoint is exactly what it is for.
- Line 3: This SELECT would run. The line before it cannot.